

Relatório de Avaliação BTreeFy — Estudo de Caso SBESC

BT (BTreeFy) vs FSM (Zephyr SMF): Eixos A, B e C

Gerado em: 29-07-2026 · **Plataforma:** Zephyr v4.3.1 · **Alvo:** `native_sim` (x86-64) **Aplicação:** Rastreador de ativos com duas variantes de funcionalidade — base (apenas rastreamento de posição) e tamper (+ detecção de violação)

Notas de Execução e Bugs Encontrados

Durante a execução do pipeline de métricas, dois bugs foram descobertos e corrigidos no próprio projeto:

[!CAUTION] **Bug 1** — `oracle.conf` continha `CONFIG_TRACKER_ORACLE_MODE=n` (deveria ser `y`). Isso fazia com que toda construção do binário `oracle` compilasse silenciosamente `main.c` (teste de fumaça) em vez de `oracle_main.c`. O marcador `ORACLE_DONE` nunca era impresso, fazendo com que todas as execuções do `oracle` e do `model_metrics.py` travassem indefinidamente esperando por ele. **Corrigido:** `oracle.conf` alterado para `CONFIG_TRACKER_ORACLE_MODE=y`.

[!CAUTION] **Bug 2** — `model_metrics.py` não reativava `CONFIG_TRACKER_TRACE` para as compilações do FSM `oracle`. As linhas de transição `FSM_TR` (usadas para reconstruir o grafo da FSM) são controladas por `CONFIG_TRACKER_TRACE`. Como `oracle.conf` define `CONFIG_TRACKER_TRACE=n`, o rastreamento da FSM ficava desabilitado e nenhuma transição era capturada, resultando em um grafo da FSM vazio e um GED de 0. **Corrigido:** `model_metrics.py` agora passa `-DCONFIG_TRACKER_TRACE=y` para a compilação do FSM `oracle`.

[!NOTE] **Eixo C** — **Compilação cruzada ARM indisponível neste workspace.** O Zephyr 4.3.1 requer o módulo `cmsis_6` para todos os alvos ARM (`cmsis_core.h`), o qual está ausente na lista de permissões do `west.yml`. Todos os alvos reais de placas ARM (nRF52840DK, Nucleo, QEMU Cortex-M3) falham com fatal error: `cmsis_core.h: No such file or directory`. Os dados de footprint foram coletados do `native_sim` (ELF host x86-64). Valores absolutos em bytes não são representativos para flash/RAM embarcadas; **as diferenças relativas entre as variantes BT e FSM continuam válidas para comparação.**

Tabela de Síntese

 Tabela de Síntese

Eixo A — Modificabilidade do Modelo

Pergunta: O quanto o modelo de decisão muda quando a funcionalidade de violação (tamper) é adicionada?

Método: Para a BT, o modelo é diretamente o arquivo XML (`models/tracker_base.xml` → `models/tracker_tamper.xml`). Para a FSM, o grafo do modelo é **reconstruído em tempo de execução** — o binário `oracle` é executado com `CONFIG_TRACKER_TRACE=y` e as linhas `FSM_TR,<t>`,

<from>, <to> são analisadas para construir um grafo direcionado das transições de estado observadas. A Distância de Edição de Grafo (GED - Graph Edit Distance) é então calculada entre as variantes base e tamper.

Dados Brutos

Variante	Nós	Arestas	CC (proxy do grafo de decisão)
bt_base	9	8	5
bt_tamper	12	11	7
fsm_base	4	6	3
fsm_tamper	5	10	6

Transição	GED
BT base → tamper	6.0
FSM base → tamper	4.0

Análise

Estrutura do modelo BT: A BT base tem 9 nós e 8 arestas, refletindo a hierarquia da árvore XML (Fallback → Sequence → [condition, Fallback → [Sequence → [GNSSFix, SendPos], RegisterFailure], Sleep]). A variante tamper adiciona 3 nós e 3 arestas (um novo Sequence no nível raiz com TamperCheck + SendTamperAlert), aumentando o GED para **6.0** — isso inclui inserções de nós mais a reestruturação das arestas no topo da árvore.

Estrutura do modelo FSM: O grafo da FSM é reconstruído a partir das transições observadas durante 500 eventos aleatórios. A FSM base produz 4 estados distintos e 6 transições (incluindo transições de/para SLEEP, GNSS_FIX, SEND_POSITION e REGISTER_FAILURE). A variante tamper adiciona 1 estado (TAMPER_ALERT) e 4 novas arestas (transições de entrada/saída de TAMPER_ALERT a partir de todos os outros estados através da macro de guarda), resultando em um GED de **4.0**.

Ponto chave: O GED da FSM (4.0) é *menor* que o da BT (6.0). Isso é, em parte, um artefato de como cada modelo é representado: o GED da BT conta edições estruturais na árvore XML (relações pai-filho), enquanto o GED da FSM conta edições no grafo de transição de estados. A extensão tamper da FSM usa uma única macro de guarda compartilhada (TRACKER_FSM_TAMPER_GUARD) injetada no início da função run() de cada estado — portanto, estruturalmente a mudança na FSM é localizada (1 novo estado, arestas de todos os estados existentes para ele). A mudança na BT reestrutura a raiz da árvore, exigindo mais operações de edição.

[!NOTE] Os valores de cc_decision_graph usam a fórmula $CC = arestas + sumidouros - nós + 1$ aplicada ao grafo bruto pai→filho ou de transição. Este é um **proxy relativo**, não o CC rigoroso dos livros-texto (que requer a transformação completa para o autômato de fluxo de controle). Use apenas para comparação direcional.

Eixo B — Modificabilidade do Código

Pergunta: O quanto o código de implementação muda quando a funcionalidade de violação (tamper) é adicionada?

Método: Duas medições nos arquivos fonte C:

1. **LOC do `#ifdef TAMPER`** — linhas não em branco dentro dos blocos `#ifdef CONFIG_TRACKER_WITH_TAMPER` (o único `#ifdef` de seleção de funcionalidade permitido neste código-fonte). Isso mede diretamente "quanto código cada lado precisou para adicionar a funcionalidade."
2. **Complexidade Ciclomática de McCabe** — calculada pelo `lizard` em todos os arquivos fonte de política, representando a complexidade estrutural do código.

Dados Brutos

Implementação	LOC <code>#ifdef TAMPER</code>	Funções	Média CC	CC máx
BT	12	8	2.25	4
FSM	30	12	2.00	4

Análise

LOC `#ifdef TAMPER` — BT: 12 vs FSM: 30 (2,5× mais para FSM)

Este é o resultado mais impressionante do Eixo B. A FSM exigiu **2,5× mais linhas de código** dentro das guardas de tamper para adicionar a funcionalidade:

- **BT (12 linhas):** Apenas as duas novas funções de ação precisaram ser envolvidas — `cond_tamper_detected()` e `action_send_tamper_alert()` em `tracker_bt_actions.c`. A estrutura da árvore em si é um arquivo XML separado sem nenhum `#ifdef`; a seleção do modelo é feita em tempo de compilação pelo CMake escolhendo o XML correto.
- **FSM (30 linhas):** Requer um novo estado (`tamper_alert_entry` , `tamper_alert_run`), uma macro de guarda (`TRACKER_FSM_TAMPER_GUARD`), e sua invocação dentro da função `run()` de cada estado existente. A entrada na tabela de estados também deve ser compilada condicionalmente. Isso espalha as mudanças por múltiplas funções em `tracker_fsm_states.c`.

Complexidade Ciclomática — aproximadamente equivalente

Ambas as implementações compartilham o mesmo CC máximo de 4 e médias de CC semelhantes (BT: 2,25, FSM: 2,00). A FSM possui mais funções (12 vs 8) porque cada estado é decomposto em funções `entry()` , `run()` e opcionalmente `exit()` separadas, enquanto a BT possui as funções de ação mais a thread de política. Nenhuma das implementações mostra uma complexidade preocupante — todas as funções estão dentro da faixa padrão de "baixa complexidade" ($CC \leq 5$).

Ponto chave: A vantagem da BT é a separação estrutural de preocupações. O modelo (o que fazer) vive no XML; o código (como fazer) contém apenas a nova ação. A FSM requer tocar em múltiplas funções existentes para adicionar transições de guarda em todos os lugares que a funcionalidade pode preempcionar.

Eixo C — Footprint Estático (Uso de Memória)

Pergunta: Qual é o custo em tamanho do binário de cada implementação e da adição da funcionalidade de violação (tamper)?

Método: Quatro compilações `native_sim` (BT/FSM × base/tamper), com `CONFIG_TRACKER_TRACE=n` para excluir a sonda de rastreamento. Tamanhos das seções ELF lidos via `pyelftools`.

[!WARNING] Estes são números do `native_sim` (Linux x86-64). Os valores absolutos **não são representativos da flash/RAM embarcada** (sem código de inicialização específico da MCU, ABI diferente, `libc` do host). As **diferenças entre as variantes são significativas** para comparação relativa; os números absolutos não.

Dados Brutos

Variante	.text (B)	.rodata (B)	.data (B)	.bss (B)	Total Flash (B)	Total RAM (B)
bt_base	30.049	6.256	1.024	1.696	37.329	2.720
bt_tamper	30.145	6.288	1.120	1.696	37.553	2.816
fsm_base	29.633	6.000	736	1.664	36.369	2.400
fsm_tamper	29.841	6.016	736	1.664	36.593	2.400

Diferenças Derivadas

Métrica	BT	FSM	Delta (BT – FSM)
Flash, base (B)	37.329	36.369	+960
Flash, tamper (B)	37.553	36.593	+960
Custo em Flash do tamper (B)	+224	+224	0
RAM, base (B)	2.720	2.400	+320
RAM, tamper (B)	2.816	2.400	+416
Custo em RAM do tamper (B)	+96	+0	+96

Análise

Base de BT vs FSM: A implementação da BT é consistentemente maior — aproximadamente **+960 bytes de flash** e **+320 bytes de RAM** na variante base. Esse overhead vem do próprio runtime da BTreefy: o motor de travessia da árvore (`btreefy.c` , `btreefy_policies.c`), o array de nós LCRS gerado a partir do XML, e o executor POSIX. Estes são custos fixos que existem independentemente do que a árvore faça.

Custo da adição da funcionalidade tamper:

- **Flash:** Ambas as implementações adicionam exatamente **+224 bytes** de flash quando o tamper está ativado. Apesar da FSM precisar de mais código C (30 LOC vs 12 LOC dentro de `ifdefs`), o tamanho do código compilado é idêntico — a macro de guarda da FSM é convertida em pequenos desvios condicionais inline, e as novas funções folha da BT são similarmente compactas.
- **RAM:** A BT adiciona **+96 bytes** de RAM (`.data` cresce de 1.024 para 1.120 bytes) para a variante tamper, provavelmente devido ao array de nós LCRS maior para a árvore tamper. A FSM adiciona **+0 bytes** de RAM — a tabela de estados é armazenada em `.rodata` e a nova entrada `STATE_TAMPER_ALERT` é compilada condicionalmente, mas o tamanho da estrutura `smf_ctx` não muda.

Ponto chave: A BT carrega um overhead de runtime fixo (~960 B flash, ~320 B RAM) em comparação com a FSM. Contudo, o **custo marginal de estender** o comportamento é comparável em flash e ligeiramente pior para a BT em RAM. Para microcontroladores restritos, a FSM permanece mais enxuta na base, mas a diferença é modesta e as vantagens de extensibilidade da BT (Eixo B) podem superar isso para aplicações que exigem frequentes mudanças comportamentais.

Eixo D — Equivalência Comportamental

Pergunta: As implementações de Árvore de Comportamento (Behavior Tree) e Máquina de Estados Finitos (FSM) produzem as mesmas exatas saídas para as mesmas entradas?

Método: O oracle (`tests/oracle/run_oracle.py`) constrói ambas as implementações e as alimenta com uma sequência deterministicamente gerada (mesma semente) de 500 eventos aleatórios (dados de sensores simulados). Ele captura seus comandos de saída e verifica a diferença (diff), alinhando-os pelo índice do evento.

Resultados

Sequência	Divergências	Inesperados	Resultado
no-tamper	0	0	✅ APROVADO
tamper	0	0	✅ APROVADO

Análise

Ambos os cenários, base (`no-tamper`) e estendido (`tamper`), produziram **zero divergências**. Isso prova que a implementação da Árvore de Comportamento é funcionalmente idêntica à implementação padrão de Máquina de Estados Finitos para este estudo de caso.

Gráficos

 Gráfico Comparativo de 4 Painéis

Resumo & Conclusões

Eixo	Métrica	BT (BTreeFy)	FSM (Zephyr SMF)	Vencedor
A	GED base→tamper	6.0	4.0	FSM
A	Crescimento em nº de nós (base→tamper)	+3 nós	+1 estado	FSM
B	LOC do <code>#ifdef Tamper</code>	12	30	BT
B	Número de funções	8	12	BT
B	Média CC	2.25	2.00	Empate
C	Flash base	37.329 B	36.369 B	FSM

C	Custo em Flash do tamper	224 B	224 B	Empate
C	RAM base	2.720 B	2.400 B	FSM
C	Custo em RAM do tamper	+96 B	+0 B	FSM
D	Equivalência Comportamental	APROVADO	APROVADO	Empate

Eixo A — Modelo: A FSM requer menos edições a nível de grafo (GED=4) que a BT (GED=6) para incorporar a funcionalidade tamper. Isso se deve ao fato da extensão tamper da FSM adicionar um único novo estado com arestas vindas de todos os estados existentes, enquanto a BT reestrutura sua raiz — uma operação de edição de árvore maior. Ambos os modelos permanecem simples e bem estruturados.

Eixo B — Código: A BT vence decisivamente. Ela exige **2,5x menos linhas de código específicas da funcionalidade** que a FSM (12 vs 30). Isso confirma a hipótese central do BTreeFy: mudanças comportamentais expressas como novos nós folha em um modelo XML exigem adições mínimas de código C, enquanto extensões em FSM devem tocar todo estado existente que pode ser preempcionado pelo novo comportamento.

Eixo C — Footprint: A FSM é mais enxuta na base (~960 B flash, ~320 B RAM). O runtime do BTreeFy é um overhead fixo que domina para árvores pequenas. O custo marginal da adição do tamper é equivalente em flash (+224 B cada) e levemente pior para a BT em RAM (+96 B vs 0 B). Para MCUs pesadamente restritas em recursos, a FSM é preferível; para aplicações onde a complexidade comportamental crescerá, o baixo custo de código por funcionalidade da BT se torna crescentemente vantajoso.

Avaliação geral do BTreeFy: O framework cumpre sua promessa principal — mudanças comportamentais são mais fáceis e baratas de expressar (Eixo B). A contrapartida é um overhead de execução fixo (Eixo C) e uma distância de edição de modelo um pouco maior quando as funcionalidades reestruturam a raiz da árvore (Eixo A). Para aplicações de rastreamento de ativos em MCUs com ≥ 256 KB de flash, o overhead de ~1 KB do BTreeFy é insignificante e a vantagem na modificabilidade do código é bastante significativa.

Avaliação de Revisor Acadêmico

O texto a seguir é uma revisão acadêmica simulada do framework BTreeFy e deste estudo de caso específico, pressupondo a submissão para uma conferência de sistemas embarcados (ex: EMSOFT, SBESC).

1. Relevância e Contribuição

É relevante como artigo acadêmico? Sim, absolutamente. A comunidade de sistemas embarcados está atualmente lidando com a crescente complexidade comportamental em dispositivos IoT e de borda (edge). Máquinas de Estados Finitos (FSMs) tornam-se famosas por serem difíceis de manter à medida que a complexidade escala (explosão de estados, espagete de transições). Propor um motor de execução de BT leve, baseado em C e feito sob medida para ambientes RTOS (como o BTreeFy) atende diretamente a um problema de engenharia de software bastante oportuno em sistemas embarcados.

2. Pontos Fortes

- **Vantagem Clara em Modificabilidade:** Os resultados do Eixo B (Modificabilidade de Código) são o argumento de venda mais forte do artigo. Provar que uma extensão comportamental (a funcionalidade tamper) requer 2,5× menos código C (12 LOC vs 30 LOC) na BT em comparação à FSM é um argumento quantitativo convincente a favor da manutenibilidade das BTs.
- **Separação Estrutural de Preocupações:** O framework prova com sucesso que as BTs permitem que a lógica/modelo (XML) seja separada da implementação (funções C). Na FSM, adicionar uma funcionalidade de preempção exigiu a modificação de uma macro de guarda que poluiu todos os estados existentes. A BT alcançou isso simplesmente adicionando um novo galho à árvore, sem modificar os nós de ação existentes.
- **Equivalência Comportamental (Eixo D):** Mostrar que a implementação da BT atinge 100% de equivalência comportamental com uma biblioteca de FSM estabelecida (Zephyr SMF) valida que o BTreeFy não é um mero experimento, mas uma alternativa viável e determinística para sistemas de produção.
- **Eixos de Avaliação Abrangentes:** A metodologia de comparação de Modificabilidade do Modelo (GED), Modificabilidade do Código (LOC/CC), Uso de Memória (Footprint) e Equivalência Comportamental é rigorosa e bem elaborada.

3. Fraquezas e Áreas de Melhoria

- **A interpretação do Eixo A (Graph Edit Distance) é fraca:** O artigo afirma que a FSM tem um GED menor (4.0) comparado à BT (6.0) para a extensão tamper. Entretanto, comparar o GED de uma árvore XML ao GED de um grafo de transição de estados é como comparar laranjas e maçãs. O GED da FSM é menor porque adicionar uma preempção global em uma FSM significa adicionar um estado e arestas de todos os outros lugares. Em uma BT, isso significa inserir um novo nó Sequence/Fallback no nível da raiz e deslocar a árvore existente para baixo. O artigo precisa discutir explicitamente *por que* o GED mais alto da BT é, na verdade, aceitável (porque ele não exige mudanças no código C da estrutura existente).
- **Overhead Fixo (Eixo C) é significativo para MCUs muito pequenas (ultra-low-end):** O runtime do BTreeFy introduz uma penalidade de ~1KB em Flash e ~320B em RAM comparado ao Zephyr SMF. Embora insignificante em um Cortex-M4 com 256KB de Flash, é um imposto significativo sobre um nó Cortex-M0+ de 16KB/32KB de Flash. O artigo deve delimitar explicitamente o hardware alvo. Ele deveria reconhecer que FSMs ainda são a escolha correta para os nós mais restritos.
- **Simplicidade do Estudo de Caso:** O rastreador de ativos usado para a avaliação é bastante simples (4-5 estados). O benefício real das BTs sobre as FSMs (evitar a explosão de estados) se torna exponencialmente mais óbvio conforme o sistema escala para 20, 30 ou 50 estados. O artigo seria muito mais forte se incluísse um gráfico projetando como a LOC e a Complexidade Ciclomática escalariam se mais 5 funcionalidades fossem adicionadas.
- **Overhead de Desempenho/Tempo (Eixo Ausente):** A avaliação cobre o footprint (memória estática), mas ignora o tempo de execução. Quantos ciclos de CPU são necessários para processar (dar um "tick") a BT da raiz até uma folha, comparado a um simples salto de ponteiro em uma FSM? Em sistemas embarcados de tempo real, o jitter de execução é crítico. Os autores deveriam incluir uma medição do pior tempo de execução (WCET - Worst-Case Execution Time) de um tick da árvore vs uma transição da FSM.

Recomendação: Aceitar (com pequenas revisões).